

Block 3

Schema Validation, Ref/Def Checks, Conformance

15 min lecture • 20 min exercise • 5 min review

Four layers, four failure classes

Layer	Catches	One-liner
1. XSD schema	Structural violations: missing required attributes or elements, bad enumeration values, wrong placement	<code>ODMSchemaValidator(...) .validate_file(path)</code>
2. OID ref/def	Dangling references, duplicate OIDs, orphan definitions	<code>odm.verify_oids(checker)</code>
3. Conformance	Model-rule violations on the object tree – works <i>before</i> you serialize	<code>odm.verify_conformance(MetadataSchema())</code>
4. Element order	Children assigned out of schema order – matters for dict/JSON output	<code>odm.verify_order()</code>

None of them subsumes another. That's the lesson of this block.

Your default call

```
errors = odm.validate(  
    collect_errors=True,  
    oid_checker=create_oid_checker("define_2_1"),  
    conformance_checker=MetadataSchema(),  
)  
for err in errors:  
    print(type(err).__name__, "-", err)
```

Runs layers **2 – 4** and collects **every** finding (*new in 0.2.1*).
Empty list means clean. `max_errors=N` caps a runaway list.

Without `collect_errors` it's fail-fast — right for pipelines,
where the exception *is* the answer.

What `validate()` misses: XSD

XSD	works on the serialized FILE
layers 2-4	work on the object TREE

Deliberate, not an oversight — the object checks run on documents you built in memory and **haven't written yet**.

You need **both**, and the exercise proves it:

- `define_broken_refs.xml` → passes XSD, **fails** layer 2
- `defineV21-SDTM-invalid-class.xml` → passes 2-4, **fails** XSD

The repair loop

```
with odmlib.permissive():      ← get a broken file into objects at all
    odm = load(path)
    |
    ▼
    odm.validate(collect_errors=True, ...) ← everything wrong, one pass
    |
    ▼
    fix the objects              ← ordinary attribute assignment
    |
    ▼
    re-validate → empty? → odm.write_xml(path)
```

Strict loading *rejects* a malformed file — correct, but useless when your job is repairing it. **This loop is the heart of every odmlib-based repair tool.**

Watch out for these

- **Reusing an OLD checker across documents** — they're stateful and single-use. Fresh one each time, or `checker.reset()`
- Calling `unreferenced_oids()` **while dangling refs remain** — it raises. Fix refs first, then hunt orphans
- Expecting `validate()` **to schema-validate** — separate layer, run both
- Fixing objects from `loader.MetadataVersion()` while validating `loader.root()` — **different trees**. Keep one `root()` reference
- Reading only the first error — use `collect_errors=True` and fix in bulk

Your turn → `validate_check_exercise.ipynb` — 4 TODOs, ~20 min